



Práctica 5. Diseñar clases para modelar el estado y comportamiento aplicando encapsulamiento, constructores y sobrecarga

Objetivo

El alumno diseñara clases y métodos para modelar la inicialización del estado y comportamiento de entidades del mundo real, encapsulando los atributos y métodos, precisando el protocolo con pase de parámetros y la asignación de valores, estructuras de control y selección con una actitud crítica, propositiva y creativa.

✂ Equipo y material de apoyo:

- Computadora Personal (PC)
- Ambiente de desarrollo o IDE (VSCODE)
- JDK Java Development Kit

DESARROLLO DE LA PRACTICA:

Para cada programa se deberá:

- Diseñar el diagrama UML de la clase.
- Aplicar correctamente el encapsulamiento (atributos privados)
- Implementar
 - Constructores
 - Sobrecarga de constructores cuando se indique
 - Metodos accedadores
 - Métodos mutadores
- Definir claramente el protocolo de los métodos (tipo de retorno, parámetros y validaciones)
- Implementar el comportamiento utilizando estructuras de control
- Compilar, ejecutar y probar con distintos valores
- Documentar el código adecuadamente

Programa 1:

Usted fue contratado por una compañía de seguros de autos que da servicio a los municipios de Baja California: Mexicali, Tijuana, Rosarito, Tecate, San Quintin y Ensenada. La compañía desea que usted



cree un programa que genere un reporte que indique si cada una de sus pólizas de seguros de automóviles se encuentra en un estado con seguro de autos “sin culpa”: Tijuana, Mexicali y Ensenada.

La aplicación de Java que cumple con estos requerimientos contiene dos clases: PolizaSeguro y PruebaPoliza.

La clase PolizaAuto representa una póliza de seguro de auto. La clase contiene: número de cuenta para almacenar el número de cuenta de la póliza, la marca y modelo para almacenar la marca y modelo del auto. La variable de instancia llamada estado para almacenar los caracteres que represente el estado en el que se encuentra la póliza. Cree un constructor que inicialice las variables de instancia de la clase, cree los métodos accedadores y mutadores. Cree un método es EstadoSinCulpa para devolver un valor boolean que indique si la póliza se encuentra en un estado con seguro de auto sin culpa. Además, diseñe el diagrama UML de la clase. Escriba el programa respectivo solicitando al usuario los elementos correspondientes.

Pruebe el programa de la siguiente manera

```
La poliza de auto:
Cuenta #: 11111111; Auto: Toyota Camry;
Estado NJ esta en un estado sin culpa

La poliza de auto:
Cuenta #: 22222222; Auto: Ford Fusion;
Estado ME no esta en un estado sin culpa
```

Programa 2: Cree una clase empleado con los siguientes atributos del cliente: nombre, dirección y teléfono. Diseñe el diagrama UML de la clase, aplique encapsulamiento e imprime los datos personales y del trabajo de tal manera que se vea presentable.

Programa 3: Escriba una clase llamada DiasDelMes. El constructor de la clase debe aceptar dos argumentos.

- Un entero para el mes (1= Enero, 2 febrero, etc)
- Un entero para el año

La clase deberá tener un método llamado getNumeroDias que regresa el número de días especificado del mes. El método debe usar el siguiente criterio para identificar años bisiestos.

Determinar si el año es divisible por 100. Si lo es, entonces es un año bisiesto si y sólo si es divisible por 400. Por ejemplo, 2000 es un año bisiesto, pero 2100 no lo es.

Si el año no es divisible por 100, entonces es un año bisiesto si y sólo si es divisible por 4. Por ejemplo, 2008 es un año bisiesto, pero 2009 no lo es.



Demuestre la clase en un programa que le pide al usuario que ingrese el mes (permitiendo al usuario ingresar un número entero en el rango de 1 a 12) y el año. El programa debería mostrar entonces la cantidad de días de ese mes. Aquí hay una ejecución de muestra del programa:

Ingrese un mes (1-12): 2

Ingrese un año: 2008

29 días

Programa 4: Cree una clase llamada Empleado, que incluya los siguientes campos:

- nombre: El campo nombre referencia un Objeto de tipo String que almacenará el nombre del empleado.
- numerold: El numerold es una variable entera que almacenará el id del empleado.
- departamento: El campo departamento referencia a un objeto de tipo String que almacenará el nombre del departamento donde trabaja el empleado.
- posición: El campo posición referencia a un objeto de tipo String que almacenará el título del trabajo que desempeña el empleado.

La clase debe contener:

- un constructor que acepte los siguientes valores como argumentos y los asigne a los apropiados campos: nombre del empleado, id del empleado, departamento y posición.
- Un constructor que acepte los siguientes valores como argumentos y los asigna a los campos apropiados: nombre del empleado y id. El departamento y la posición deberán ser asignados con una cadena vacía (" ").
- Un constructor con no-argumentos que asigna cadenas vacías (" ") al nombre, departamento y posición, y 0 al id.

Escribe los métodos mutadores que almacenarán los valores en los campos y los métodos accedores que regresarán los valores de esos campos. Una vez escrita la clase, escribe por separado un programa que crea tres objetos Empleados que contengan los siguientes datos:

Name	ID Number	Department	Position
Susan Meyers	47899	Accounting	Vice President
Mark Jones	39119	IT	Programmer
Joy Rogers	81774	Manufacturing	Engineer

El programa deberá guardar estos datos en tres objetos y después desplegar los datos para cada empleado en la pantalla.

Entrega:



- La entrega se realizará mediante una actividad en BB. Se han de entregar los archivos fuentes desarrollados, más el reporte de la practica adjuntos adjuntos en un fichero zip cuyo nombre esté formado por el número de práctica y nombre del alumno. Ejemplo Practica4_JuanLopez.pdf
- Hacer una clase por cada ejercicio y adjuntarlos
- Entregar un reporte de práctica, formato reporte técnico y electrónico, podrá utilizar este formato y agregar el procedimiento correspondiente), que conceptos fundamentales se aplicaron en esta practica, junto con resultados y conclusiones.

Análisis de Resultados

Conclusiones

Referencias

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, titulo, la descripción de la práctica, nombre del alumno que realizo la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

- | | |
|---|-------------|
| • Portada, competencia u objetivo | 2.5 puntos |
| • Pegar Código del programa (anexar archivos .java) | 5 puntos |
| • Explicación completa del código y su funcionamiento | 12.5 puntos |
| • Resultados (análisis de resultados) | 5 puntos |
| • Conclusiones: | 5 puntos |

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados, serán anulados automáticamente para ambos alumnos.

El reporte debe contener lo anterior mencionado y quitar las instrucciones, el reporte si es leído por cualquier persona, esta debe ser capaz de repetir todo el proceso sin errores (por lo que debes de incluir toda la información).



La conclusión no debe ser describir el proceso que se siguió, investigar que debe contener una conclusión.